BaiZe Technical Report

***Abstract*—Large language models (LLMs) exhibit strong reasoning and tool-use skills, yet their computational footprint makes most agentic models impractical for edge, embedded, and cost-sensitive electronic design automation (EDA) deployments. We present BaiZe, a ≈2.2-billion-parameter agentic language model built *from scratch* on a fixed compute budget around a *hybrid* state-space backbone. BaiZe adopts a sequential hybrid architecture—56 layers interleaving Mamba-2 state-space layers, four grouped-query-attention layers, and MLP layers—whose linear-time decoding is well suited to inference-heavy, cost-sensitive agents, and is trained with the NVIDIA NeMo recipe on Megatron-Core. The architecture is chosen through a matched from-scratch comparison against a dense transformer counterpart, in which the hybrid attains lower loss and faster inference with fewer parameters at a modest training-throughput cost. We then run a staged, budget-constrained hyper-parameter search (S1–S5) on $8\times$H100 over the final hybrid backbone, which selects a stable learning rate of 1×10^{-3} under a warmup–stable–decay schedule with an $86{:}10{:}4$ annealing mixture, reaching a validation loss of $\mathbf{2.2054}$ after 20000 steps and 2.6739 ± 0.0469 across three seeds at 5000 steps. The search recipe is then scaled to a full production run of ≈44B tokens—the Chinchilla-optimal budget for a 2.22B model—reaching a final training loss of $\mathbf{1.357}$ in ≈412 GPU-hours on $8\times$H100. We release the complete recipe and training command to support the reproducible construction of efficient agentic small language models.**

***Index Terms*—small language model, agentic LLM, hybrid architecture, state-space model, budget-constrained search, efficient inference**

I. Introduction

Large language models have recently demonstrated agentic capabilities—planning, tool invocation, and long-horizon reasoning—that make them attractive cores for intelligent assistants spanning chip design, verification, and other domains of electronic design automation. However, state-of-the-art agent backbones are routinely 7 B parameters or larger, demanding high-end accelerators and memory footprints that are incompatible with edge, embedded, and cost-sensitive deployments where inference latency and energy are first-class constraints. The constraint is sharper than raw cost in the settings we target: proprietary design artifacts—layouts, schematics and review judgements—cannot leave the premises, so an agentic core must be served locally; and because an agentic session accumulates context, what dominates serving cost is not prompt throughput but the memory that grows with the session. For a hybrid backbone this is architectural: only the four attention layers carry a session-growing state, while the Mamba-2 layers hold a constant-size recurrent one.

Recent small language models (SLMs, <2 B) have closed much of the gap with larger counterparts on single-turn benchmarks, but building such a model *from scratch* remains an under-documented engineering problem. Every choice—architecture, optimizer, learning-rate schedule, data mixture—interacts with every other, and a naive grid search is prohibitively expensive. Our goal is therefore twofold: (i) to construct, from scratch and on a strictly limited compute budget, an end-to-end multimodal agentic system—an efficient hybrid language core paired with a custom vision encoder, aligned into a semiconductor-domain agent—and (ii) to expose the concrete, reproducible decisions that lead to it.

a) BaiZe.: We present BaiZe, an end-to-end multimodal agentic system assembled from scratch across five stages: (i) pre-training a ≈2.2B-parameter hybrid (state-space + attention) LLM, (ii) post-training it for agentic capability, (iii) pre-training a custom vision encoder, (iv) aligning the two into a multimodal model, and (v) adapting the result to the semiconductor domain. Three design pillars organise our work:

- **Architecture.** A sequential hybrid language backbone—56 layers interleaving Mamba-2 state-space layers, four grouped-query-attention layers, and MLP layers—giving linear-time decoding for inference-heavy agents, paired with a from-scratch vision encoder.
- **Reduced-horizon proxy search.** We exploit short, cheap proxy runs to rank candidate hyper-parameters, so that the ranking discovered at small scale carries over to the full model without re-tuning at scale.
- **Budget-constrained stage-wise pipeline.** Each of the five stages is trained under a hard GPU-hour budget, with a staged search protocol (S1–S5) for the language backbone and a single frozen evaluation protocol shared by the later stages.

We hope BaiZe serves as a minimal, reproducible blueprint for building efficient agentic SLMs from scratch, and that our search methodology is of practical value to cost-bounded training efforts in the EDA and embedded-AI communities.

II. Background and Related Work

A. Small Language Models and Agentic Reasoning

Parameter-efficient SLMs (<2 B) have recently narrowed the gap with larger models on mathematical and commonsense reasoning. MiniCPM [1] introduced scalable training strategies and domain-weighted data mixing for small models; Qwen [2] and Gemma [3] demonstrate that strong code- and math-enriched corpora can lift small models substantially. Most of these efforts, however, treat the final trained model as the deliverable and report comparatively little about the *search* that produced it. Complementarily, agentic instruction-tuning datasets (e.g., tool-use and multi-step reasoning corpora) [4] have been shown to elicit planning and tool-invocation behaviour, yet are rarely paired with a from-scratch pre-training recipe for the 1B regime.

B. Hybrid and State-Space Architectures

State-space models (SSMs) such as Mamba [5] and Mamba-2 [6] replace full attention with a recurrent scan update, offering linear-time decoding that suits inference-bound deployments. Hybrid designs interleave a small number of attention layers among SSM/MLP layers—notably Nemotron-H [7] and Hymba [8]—to recover long-range context while retaining fast decode. Released Megatron-Core variants of these families are scarce below 2B, leaving their small-scale trade-offs underdocumented; our controlled 2B comparison (Section IV-A) addresses precisely this gap.

C. Multimodal Vision-Language Models

Vision-language models connect a vision encoder to a language model through a lightweight projector, enabling end-to-end visual understanding and instruction following. LLaVA [9] established visual instruction tuning—aligning a vision encoder to a frozen LLM via a linear projection followed by instruction fine-tuning—as a scalable recipe for open multimodal assistants. LLaVA-OneVision [10] consolidates this line into a single open model spanning single-image, multi-image, and video tasks through a staged curriculum: stage-1 projector pre-training, stage-1.5 mid-training, and stage-2 instruction tuning. BaiZe follows this curriculum in stage (iv) (Section VII), substituting a from-scratch vision encoder (stage iii) for the frozen off-the-shelf encoder.

D. LLMs for Electronic Design Automation

A growing line of work applies LLM agents to chip design tasks such as register-transfer-level generation, verification, and bug triage [11], [12]. These applications motivate a cheap, always-on, agentic multimodal core that can run per-team or per-engineer; our work supplies the efficient multimodal backbone such systems require.

III. Architecture and Pipeline Overview

BaiZe is an end-to-end multimodal agentic system assembled in five stages: (i) from-scratch pre-training of a hybrid language backbone, (ii) post-training of that backbone (instruction tuning and reinforcement learning to strengthen agentic capability), (iii) pre-training of a custom vision encoder, (iv) alignment of the two into a multimodal large language model, and (v) semiconductor-domain adaptation via reinforcement learning. This section fixes the shared scaffolding—the two backbones, the overall pipeline, and the global training environment—before each stage is detailed in its own section.

A. Language Backbone

The language backbone is a sequential hybrid model [6], [7]: 56 transformer layers interleave Mamba-2 state-space layers, four grouped-query-attention (GQA) layers, and MLP layers. The layer allocation follows the Nemotron-H sequential hybrid convention—four attention layers among the 56, the remainder Mamba-2 and MLP—as provided by Megatron-Core's Mamba-hybrid recipe, whose 56-layer pattern we reuse verbatim while halving only the hidden and FFN widths to reach the ≈2B scale. BaiZe is not a Nemotron-H model: it is an order of magnitude smaller, contains no mixture-of-experts layers, and is pre-trained from scratch on an independent corpus under a fixed GPU-hour budget. Each Mamba-2 layer uses 8 state-space groups; the four attention layers use 16 query heads and 4 query groups (GQA) with untied word embeddings. Crucially, only the four attention layers carry rotary position embeddings (RoPE); the 52 Mamba-2 state-space layers are position-agnostic, so the backbone's long-context extrapolation quality depends on just $4/56{=}7.1\%$ of layers, an asymmetry exploited in Section IV-G. The full configuration is listed in Table I; the model totals ≈2.22B parameters.

TABLE I
BaiZe language-backbone configuration (Mamba2-hybrid).

Hyper-parameter	Value
Transformer layers	56
Architecture	hybrid (Mamba-2 + GQA + MLP)
Attention layers	4 (interleaved)
Hidden size	2048
FFN hidden size	8192
Attention heads (Q)	16
Query groups (GQA)	4
Mamba-2 state-space groups	8
Vocabulary size	129,281
Sequence length	4096
Activation	SiLU / SwiGLU
Normalisation	RMSNorm
Precision	BF16
Parameters	≈2.22B

TABLE II
BaiZe five-stage pipeline overview.

Stage	Task	Status	Section
(i)	LM pre-training	complete	IV
(ii)	Post-training (SFT, RLVR)	design	V
(iii)	Vision encoder	complete	VI
(iv)	MLLM alignment	design	VII
(v)	Domain adapt. (RLHF)	design	VIII

B. Vision Encoder

The vision encoder is pre-trained from scratch in stage (iii) and is selected from six iso-parameter candidates at the 500–600 M scale; Section VI reports the comparison and its outcome.

C. Pipeline and Global Environment

Table II summarises the five-stage pipeline. All stages run on 8×NVIDIA H100 GPUs (CUDA 12.8, PyTorch 2.8.0, Python 3.10); fused kernels, selective activation recomputation, and cross-entropy loss fusion are enabled where applicable, with `tensor_model_parallel_size = 1` and BF16 precision throughout.

IV. LLM Pretraining

Stage (i) pre-trains the hybrid language backbone (Section III-A) from scratch under a fixed budget.

TABLE III
MEASURED DOCUMENT-LENGTH DISTRIBUTION OF EN_V1_4 (8 SHARDS, 8.59B TOKENS, 9.66M DOCUMENTS).

Threshold	Docs	Token share	Status vs. target
≥32K	4,257	**2.52%**	target ≥20%: **not met**
≥128K	—	**0.03%**	target ≥5%: **not met**
≥1M	—	not estimated	not required (extrapolated)

TABLE IV
PRODUCTION BUDGET. WALL-CLOCK IS ESTIMATED AT MEASURED THROUGHPUT.

Phase	Tokens	Steps	Wall-clock	Status
Warmup + stable	2.94T	700k	≈56 d	30.7k done
Decay A (16K, 60–70%)	58.7B	14k	≈1.2 d	queued
Decay B (LC, 30–40%)	25.2B	6k	≈2.3 d	gated
Total	**3.02T**	**720k**	≈59.5 d	—

A. Architecture Selection

We adopt the hybrid backbone for two reasons: state-space layers decode in constant time per token while the four attention layers retain long-range context, and a matched from-scratch comparison at the ≈2B scale against a dense transformer (MiniCPM5-2B) favours the hybrid, which reaches lower loss with 11.6% fewer parameters (≈2.22B vs. 2.51B) at a ∼8% training-throughput cost (Table VI).

B. Training Framework and Learning-Rate Schedule

We train with the NVIDIA NeMo [13] entry point driving Megatron-Core's [14] Mamba-hybrid provider at `tensor_model_parallel_size` = 1, with fused kernels and selective activation recomputation enabled. All hyperparameter searches run at a reduced iteration count; the short-horizon ranking is transferred to full scale. BaiZe trains under a warmup–stable–decay (WSD) schedule with a stable learning rate of 1×10^{-3} and 2,000 warmup steps. The main training sequence length is 16,384: this is the throughput-optimal operating point (613,992 tok/s at CP=1/DP=16; measured) and it keeps the extrapolation ratio to the 1M product target at 64× rather than the 256× a 4K length would imply. The decay phase has two segments: segment A anneals at 16K on instruction and curated web data (60–70% of the decay budget), and segment B extends the length on a 32K→64K→128K ladder (30–40%). Decay follows $lr(p) = lr_{\text{peak}}(lr_{\min}/lr_{\text{peak}})^p$ with $lr_{\min}=1\times10^{-4}$ (Table IV).

C. Long-Context Data Gate

Segment B learns whatever length structure the corpus contains, so we measured the document-length distribution before budgeting it: only **2.52**% of tokens sit in documents of ≥32K and 0.03% reach ≥128K, with a mean document of 889 tokens (Table III). Training a 128K rung on this distribution would teach document concatenation rather than long-context retrieval, so we impose three entry conditions: supplement long documents until the ≥32K share reaches 20% and the ≥128K share reaches 5% (until then the ladder stops at 64K); reset attention and Mamba-2 state at every EOD; and de-duplicate long documents first. Because a full grid is infeasible, the recipe was selected with a staged, single-variable-at-a-time protocol at a 5000-step horizon: S1 baseline; S2 learning-rate sweep (2×10^{-4}–1×10^{-3}, seven points); S3 schedule axes (WSD vs. cosine, decay ratio, warmup, min_lr); S4 annealing data mix; S5 long run plus three seeds. The total search budget is ≈92 GPU-hours on eight H100s, and the reported losses are proxy measurements for ranking, not production estimates.

D. Data

The stable phase draws on a seven-source mixture—`en_v1_4` (0.65), `ufw_l1_en_hq` (0.05), `ufw_zh` (0.04), `math_l1` (0.05), `math_l3` (0.02), `code_l2` (0.15), and `code_l3` (0.04)—which keeps an 88:8:4 web/code/math prior across `Ultra-FineWeb`, `UltraData-Code`, and `UltraData-Math`. Sources are pre-blended into large shards so each rank opens $\mathcal{O}(10)$ index files instead of $\mathcal{O}(600)$, which bounds the data-loader footprint and is a precondition for the throughput above. Contamination is controlled by a 193,295-entry 13-gram fingerprint blacklist built from all 536 benchmark tasks; every shard is scanned for ≥0.80 n-gram overlap before release.

E. Results and Production Run

The search selects a stable learning rate of 1×10^{-3} under WSD with a 86:10:4 annealing mixture, reaching a validation loss of $\mathbf{2.2054}$ at 20000 steps and 2.6739 ± 0.0469 across three seeds at 5000 steps (Table VII). Each ablation confirms a single-variable choice: WSD beats cosine (2.7679 vs. 2.8155), a 10% decay ratio beats 5%, and 250 warmup steps beat 2000. A mixture search over 400 Bayesian trials at 18.36M proxy scale found the three-way ranking indistinguishable within 0.6 pp, so the existing 88:8:4 prior is retained rather than re-tuned. The recipe is scaled to 2.94T tokens in the warmup-plus-stable phase at TP=1/CP=1/DP=16 over two 8×H100 nodes, micro-batch 1 at sequence 16,384 (GBS=256, 4,194,304 tokens/step), FP8 tensorwise, computing-graph replay disabled, AdamW. Replay is disabled deliberately: a replayed graph over the data-dependent Mamba-2 state hit a throughput plateau, and this is a precondition for the rate below. At the time of writing (31 October 2026) the run has passed iteration 30,660 of 700,000 at a sustained ≈598K tok/s with training loss 2.151, gradient norms stable at 0.09–0.10, and no NaN; the remaining schedule is projected at the measured rate (Table IV). Precision was chosen per configuration rather than globally: at sequence 4096 the FP8 tile (M=8192) sits below the crossover point and runs at 0.91× BF16, whereas at M=16,384 FP8 pays, and the 613,992 tok/s peak is an FP8 measurement.

F. Long-Context Training in the Decay Tail

Long-context training is segment B of the decay phase, not a separate stage and not part of SFT/RL: the learning rate

TABLE V
LONG-CONTEXT LADDER. TOKENS PER GPU ARE HELD AT 16,384; 32K THROUGHPUT IS MEASURED, 64K/128K EXTRAPOLATED.

Rung	Seq.	CP / DP	GBS	Tokens	Throughput
1	32K	2 / 8	128	≈42B	351,526
2	64K	4 / 4	64	≈32B	≈180K (est.)
3	128K	8 / 2	32	≈32B	≈90K (est.)

is already low, so the length-distribution shift perturbs near-converged weights rather than peak-LR weights; and neither SFT (which shapes instruction following) nor RL (which optimises a policy preference) establishes ground retrieval ability at the token scale this requires. The decay phase is budgeted at 10–15% of the main training volume (294–441B tokens), of which long-context data takes 30–40% (88–176B), split 40–50%/30%/20–30% over the three rungs so that most tokens are spent where the model is weakest . Synthetic needle data takes 20–40% of the long-context tokens and is diversified over single-needle NIAH, multi-needle MK-NIAH (all-hit scoring), multi-hop tracing, variable-length needles, and paraphrase-plus-noise; the remainder is natural long documents, since a purely synthetic segment teaches answering about inserted needles rather than reading documents. The ladder runs 32K→64K→128K with $\texttt{CP} \in \{2, 4, 8\}$, holding $\texttt{seq}/\texttt{CP} = 16{,}384$ per GPU and 4.19M tokens/step at every rung (Table V). The learning rate continues the decay and is never re-warmed; $\theta{=}10^6$ is fixed at initialisation, so 128K is reached by training and 1M is only an $8\times$ extrapolation; no position interpolation, NTK rescaling, or YaRN is enabled. The reference rung-cycle costs ≈118 GPU-hours on sixteen GPUs, carried by context parallelism rather than extra nodes. When the WSD schedule ends we take one snapshot and measure base capabilities—common-sense reasoning, needle retrieval (passkey and RULER, target ≥95% at 1M), perplexity extrapolation ($\leq 1.10\times$ at 1M), and basic format following—reported separately from post-training. Retrieval uses a depth-by-position grid with all-hit scoring, because perplexity alone is not accepted as evidence of long-context ability; complex reasoning and agentic ability are recorded as a baseline only.

G. Long-Context Capability and Inference Cost

With adjusted base frequency ($\theta{=}10^6$) on the four attention layers, perplexity evolves $83.2 \rightarrow 51.5 \rightarrow 53.3 \rightarrow 53.0 \rightarrow 55.8 \rightarrow 55.4$ from 4K to 1M, i.e. flat beyond 64K; the stability follows because only 4 of 56 layers carry RoPE, so the extrapolation burden falls on 7.1% of the network. Under the SGLang production stack, prefill stays at 41.6K tok/s up to 64K and serves 1M in 172s at 34.7 GB, whereas the dense transformer cannot serve beyond 512K on a single 80 GB H100 (KV pool cap ≈455K tokens); the hybrid's advantage is architectural, since the Mamba-2 state grows in $O(1)$ for 52 of 56 layers.

TABLE VI
ARCHITECTURE-LEVEL COMPARISON MOTIVATING THE HYBRID CHOICE (DENSE = MINICPM5-2B, HYBRID = BAIZE; MEASURED FROM SCRATCH UNDER MATCHED DATA, SEED, AND GPU FOOTPRINT, SAME NODE, SAME CONFIGURATION). TRAINING THROUGHPUT WAS RE-MEASURED ON A SINGLE 8×H100 NODE AFTER AN INITIAL CROSS-NODE MEASUREMENT WAS FOUND TO OVERSTATE THE GAP. THE DECODE GAP IS MEASURED UNDER THE MEGATRON-CORE DIRECT-DRIVE HARNESS WITHOUT CUDA GRAPHS, WHERE THE DENSE BASELINE IS CPU-LAUNCH-BOUND, AND THEREFORE NARROWS UNDER PRODUCTION INFERENCE STACKS ().

Metric	Dense	Hybrid	Edge
Parameters	2.51B	**2.22B**	hybrid (−11.6%)
Final loss (1k steps)	4.81	**4.68**	hybrid (−0.12)
Train loss (5k steps)	3.36	**3.06**	hybrid (−0.30)
Training tok/s	**90K**	83K	dense (+8%)
Prefill tok/s	21.4K	**23.7K**	hybrid (+10.5%)
Decode tok/s	2.2	**22.8**	hybrid (+10.3×)

TABLE VII
HYPER-PARAMETER AND DATA-MIX SEARCH OUTCOMES AT A MATCHED 5000-STEP (S5: 20000-STEP) HORIZON.

ID	Change vs. best	Val. loss	Outcome
S1-01	baseline ($lr{=}3{\times}10^{-4}$, WSD)	2.849	baseline
S2-07	stable LR $= 1 \times 10^{-3}$	**2.7627**	best LR
S3-02	schedule family: cosine	2.8155	WSD retains
S3-03	decay ratio 5%	2.7900	10% retains
S3-04	warmup 2000 steps	2.7957	250 retains
S4-03	anneal L3+code+math 86:10:4	**2.6298**	best mix
S5-01	winner, 20000 steps	**2.2054**	**winner**

V. LLM POST-TRAINING

Stage (ii) adapts the pre-trained backbone to EDA scripting, and it is the stage that carries BaiZe's domain claim: the pre-training corpus of Section IV-D is general web, code, and mathematics text and contains no EDA artifacts, so the domain prior must be introduced here. We organise the stage as three steps—*domain annealing*, *supervised fine-tuning*, and *reinforcement learning*—all evaluated under a single frozen protocol (Section V-D).

A. Domain Annealing

The cheapest lever on the domain prior is to continue pre-training on EDA text before any behavioural tuning. We reuse the warmup–stable–decay machinery of Stage (i) and anneal on a corpus assembled from PyAether and SKILL API references, EDA tool documentation, and open HDL sources, blended with the general mixture at a controlled ratio. Because the step changes only the data distribution, we contrast it against an SFT-only control to isolate its contribution from the behavioural tuning that follows.

B. Supervised Fine-Tuning

SFT teaches the generate–execute–refine loop that the deployment target requires. The corpus combines the agentic instruction set (`UltraData-SFT-Agent-2609`, 51 G across 50 shards) and a general instruction set (`UltraData-SFT-2605`, 319 G across 1,504 JSONL files,

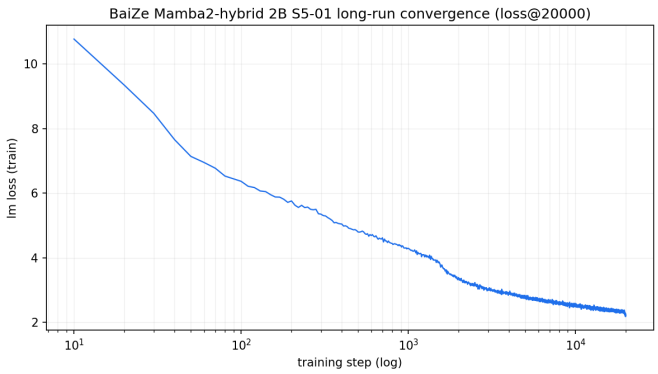


Fig. 1. Training-loss curve of the winning Mamba2-hybrid configuration over the 20000-step verification run (seed 1234); the decay tail (steps 19500–20000) anneals the loss from ~ 2.34 to 2.2054.

TABLE VIII
REWARD VARIANTS STUDIED IN STAGE (II).

ID	Reward
R1	binary pass/fail
R2	executable + assertion pass ratio
R3	R2 + tool-call penalty
R4	R3 + API-legality term

verified byte-identical to the HuggingFace manifest) with PyAether-specific material synthesised from API documentation. We ablate the data mixture (agentic / code / mathematics / PyAether), the presence of interleaved tool-call traces, full fine-tuning versus LoRA, and the sequence length: agentic traces—code, tool returns, and successive refinements—routinely exceed the 4096-token window of Table I, so window extension is treated as a first-class variable rather than a fixed setting.

C. Reinforcement Learning with Verifiable Rewards

An execution sandbox turns correctness into a verifiable signal, so we optimise with group-relative policy optimisation and no value network. Table VIII separates the reward variants we study. A binary signal is the simplest baseline; partial credit over executable attempts addresses the sparsity of pass/fail; a tool-call penalty aligns the policy with the harness cost model; and an API-legality term penalises hallucinated symbols. Because the reward is computed from assertions a policy could in principle target directly, we keep the harness anti-cheat hook enabled in every run and report a cheat rate beside every accuracy figure.

D. Evaluation Protocol

We track four capability milestones through Stage (ii): **M1** tool-call legality (syntactically valid PyAether calls, no hallucinated symbols); **M2** single-task pass (Pass@1 on EDA-Eval-PyAether); **M3** multi-turn refinement (recovery from execution failures with up to 3 rounds); **M4** full-benchmark accuracy. All variants are scored on **EDA-Eval-PyAether** [15] (158 real-world EDA scripting tasks), each pairing a natural-language prompt and function signature with assertion-based test code, scored by execution in the vendor sandbox. We follow the benchmark's own protocol—five independent runs, reshuffled order, fresh context, mean Pass@1 with run-to-run standard deviation, 2500-s per-task timeout—so numbers are comparable to published ones. Two controls make the comparison interpretable: contamination is excluded by the n-gram blacklist of Section IV-D, and only the backbone is varied while the retrieval stack, sandbox, anti-cheat hook, and agent scaffold are held byte-identical, since the harness supplies most of the observed accuracy.

Results are reported as **[TBD]**.

VI. Vision Encoder Pretraining

Stage (iii) pre-trains a custom vision encoder from scratch (Section VII); we summarise the outcome here and defer the design study to a companion report.

a) Selected encoder.: We adopt **OpenVision2**, a width-1024/depth-30 pure-attention ViT with 126.8M parameters (505.2M at full width), trained from scratch with a bidirectional trunk at 224px/16-px patches on a ≈18.5M-pair image–text corpus (CC12M + Amshaker). The objective combines a global InfoNCE contrastive term with a caption-*independent* masked-patch reconstruction term at a 1:1 weight and a 0.6 mask ratio. Both terms are required: the dense term carries most of the representation quality, while the contrastive term acts as an anchor without which the trunk degrades.

b) Final result.: On the frozen trunk with an ImageNet-1k linear probe, the encoder reaches $11.50\% \rightarrow \mathbf{19.76}\%$ top-1 from 5.1M to 55.3M samples, and 20.27% at 176M samples with no saturation. This is $+12.4$ pp above a contrastive-only baseline at the same sample budget, and it is the operating point used by stage (iv). The $224/16$ resolution is retained because it is best on both loss and throughput; whether 196 visual tokens suffice for detail-dense layout and schematic views is tested in stage (iv) (Section VII).

VII. MLLM Alignment

Stage (iv) connects the vision encoder of Section VI to the language backbone through a lightweight projector, following the staged curriculum established for open multimodal assistants: projector pre-training, mid-training, and instruction tuning. The design is motivated by the observation that the dominant EDA artifacts are visual—layout and schematic views account for 70.2% of the tasks in the evaluation benchmark [15]—so a multimodal core can be scored on the same execution-based protocol as the text-only one.

a) Task construction.: We build three task families, each chosen because it admits an objective ground truth. (i) *image→script*: given a rendered layout or schematic view, produce the PyAether or SKILL script that recreates it; supervision is available because executing a script yields its own reference rendering. (ii) *script→image*: predict the geometry that a given script produces, again self-labelled by execution. (iii) *layout VQA*: answer attribute questions—layer identity,

device type, spacing and rule violations—labelled by domain experts. The first family is the strongest of the three: its output is a script, so it is scored by the same assertion sandbox used in stage (ii), which gives the multimodal and text-only stages a common metric instead of two incommensurable ones.

b) Curriculum and ablations.: We run stage-1 projector pre-training on caption-style pairs, stage-1.5 mid-training on the three families above, and stage-2 instruction tuning on the full mixture. The ablations vary the projector (MLP versus query-based), whether the vision tower is frozen or updated, and the number of visual tokens per image. The last is of particular interest. The architecture study of Section VI selected $224/16$, where an image becomes only $14\times14 = 196$ tokens ($\sim16\times16$ px/token)—a regime in which the training objective is dominated by coarse semantics. For detail-dense line drawings, a 336-px view would yield 576 tokens ($2.9\times$ more) and a 448-px view 1024 tokens ($5.2\times$ more); whether the 196-token setting transfers to such views is precisely what stage (iv) tests. Layout and schematic views are the opposite: they are detail-dense line drawings whose meaning lives in fine geometry. We therefore treat resolution and visual-token count as first-order variables here rather than inheriting the earlier null result, which we read as a property of the proxy task and not of the encoder.

Results are reported as **[TBD]**.

VIII. Semiconductor Domain Adaptation

Stage (v) completes the pipeline by adapting the multimodal model to proprietary design practice. It differs from stage (ii) in two respects, which we state explicitly because otherwise the two stages would coincide: the reward is no longer verifiable but *learned from human judgement*, and the target behaviour is no longer general API usage but a team's internal conventions—naming rules, layer policies, review checklists—which no public benchmark measures.

a) Method.: We collect pairwise judgements from in-house design reviews over the stage-(iv) task families, fit a domain reward model on them, and optimise the aligned model against that model under a KL-regularised objective. Because preference data is far scarcer than the verifiable signal of stage (ii), we initialise from the stage-(iv) checkpoint and constrain the update rather than training the objective from the pre-trained model.

b) Why a small model.: The rationale for a ≈2.2B core is strongest here, and it is a constraint rather than a preference. Proprietary layouts and proprietary preference data cannot leave the premises, so the choice is not between a large hosted model and a small local one—it is between a small local model and no model at all. Serving cost then dominates, and for an agentic session the relevant cost is not throughput on a short prompt but memory that grows with accumulated context: in the hybrid backbone of Section III-A, the four attention layers are the only component whose serving state grows with the session, while the Mamba-2 layers hold a constant-size recurrent state.

c) Evaluation.: We evaluate on held-out review items using expert agreement, and report the result beside the execution-based numbers of stage (ii) so that verifiable and preference-based adaptation can be compared on the same system rather than in separate accounts.

Results are reported as **[TBD]**.

IX. Conclusion

We presented BaiZe, an end-to-end multimodal agentic system: a ≈2.2B-parameter hybrid (state-space + attention) language model built from scratch, post-trained for agentic capability, paired with a custom vision encoder, aligned into a multimodal model, and adapted to the semiconductor domain. Through a controlled from-scratch comparison, we selected a sequential Mamba2-hybrid backbone over a dense transformer for its lower loss-per-parameter and substantially faster inference, then applied a staged, budget-constrained hyperparameter search (S1–S5) on the final backbone. Stages (ii) to (v) are specified here as a protocol and evaluated on a common execution-based benchmark [15]; their measurements are reported as **[TBD]**. The complete recipe and a single reproducible command encapsulate the entire construction under the NVIDIA NeMo recipe.

The immediate next step is agentic fine-tuning with the `UltraData-SFT-Agent` corpus toward a deployable agent core for EDA and embedded applications. We hope this blueprint lowers the barrier for cost-bounded teams building their own efficient agentic SLMs from scratch.

References

[1] S. Hu, Y. Tu, X. Han, C. He, G. Cui, X. Long, Z. Zheng, Y. Fang, Y. Huang, W. Zhao *et al.*, "Minicpm: Unveiling the potential of small language models with scalable training strategies," *arXiv preprint arXiv:2404.06395*, 2024.

[2] A. Yang, B. Li, B. Zhang *et al.*, "Qwen3 technical report," 2025.

[3] G. Team, "Gemma 3 technical report," 2025.

[4] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang *et al.*, "Agentbench: Evaluating llms as agents," in *International Conference on Learning Representations (ICLR)*, 2024.

[5] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," *arXiv preprint arXiv:2312.00752*, 2023.

[6] T. Dao and A. Gu, "Transformers are ssms: Generalized models and efficient algorithms through structured state space duality," *arXiv preprint arXiv:2405.21060*, 2024.

[7] NVIDIA, "Nemotron-h: A family of accurate and efficient hybrid mamba-transformer models," 2025.

[8] X. Dong, Y. Fu, S. Diao *et al.*, "Hymba: A hybrid-head architecture for small language models," 2024.

[9] H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual instruction tuning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[10] B. Li, Y. Zhang, D. Guo, R. Zhang, F. Li, H. Zhang, K. Zhang, P. Zhang, Y. Li, Z. Liu, and C. Li, "Llava-onevision: Easy visual task transfer," 2024.

[11] M. Liu, T. E. Ene, R. Kirby, C. Cheng, N. Pinckney, R. Liang, J. Alben, H. Anand, S. Banerjee, I. Bayraktaroglu *et al.*, "Chipnemo: Domain-adapted llms for chip design," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[12] J. Blocklove, R. Chapman, B. Tan, D. Chen *et al.*, "Chipbench: The llm chip design benchmark," 2024.

[13] NVIDIA, "Nemo: A toolkit for building ai applications using neural modules," https://github.com/NVIDIA/NeMo, 2024, version 2.x; used for pre-training via the NeMo run entry point.

[14] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-lm: Training multi-billion parameter language models using model parallelism," 2020.

[15] Y. Liu, S. Hou, X. Chen, Y. Wang, S. Yuan, Q. Gan, S. You, F. Chen, W. Li, S. Hu, Y. Liu, E. Xia, X. Lu, H. Wang, F. Xu, and Y. Li, "Zhulong: Execution-grounded llm agent for eda scripting with offline api self-exploration," 2026.